

SmartCow Tracker

Sistema de Rastreo y Monitoreo de Ganado Bovino

SOFTWARE DESIGN DOCUMENT (SDD)

IEEE Std 1016-2009 · SOLID Principles · Clean Architecture

Versión	1.0.0
Fecha	Mayo 2026
Basado en	IEEE Std 1016-2009
Estado	Borrador — Revisión inicial

Historial de Revisiones

Versión	Fecha	Descripción	Autor
1.0.0	28/05/2026	Versión inicial SDD — SmartCow Tracker	Equipo SmartCow

1. Introducción

1.1 Propósito

El presente Software Design Document (SDD) describe el diseño detallado del sistema SmartCow Tracker, conforme al estándar IEEE Std 1016-2009 (IEEE Standard for Information Technology — Systems Design — Software Design Descriptions). Su objetivo es traducir la arquitectura definida en el SAD al diseño concreto de cada componente: interfaces de programación, contratos de datos, lógica de negocio, patrones de diseño aplicados, esquema de base de datos y diseño de las APIs del sistema.

Este documento es la referencia técnica principal para el equipo de desarrollo durante la fase de implementación. Toda decisión de implementación que difiera de lo aquí especificado debe ser registrada como un ADR (Architecture Decision Record).

1.2 Alcance

El SDD cubre el diseño interno de los siguientes módulos del sistema: Autenticación y Seguridad, Rastreo GPS en Tiempo Real, Geocercas, Inventario de Ganado, Sensores IoT, Motor de Alertas, Analítica e IA, y el diseño de la API REST completa. Incluye también el diseño del esquema de base de datos PostgreSQL, los contratos de datos (DTOs), los patrones de diseño aplicados y los principales algoritmos del sistema.

1.3 Referencias

- IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.
- SmartCow Tracker SRS v1.0.0 — Software Requirements Specification.
- SmartCow Tracker SAD v1.0.0 — Software Architecture Document.
- Martin, R. C. (2017) — Clean Architecture: A Craftsman's Guide to Software Structure and Design.
- Gamma, E. et al. (1994) — Design Patterns: Elements of Reusable Object-Oriented Software (GoF).
- Fowler, M. (2018) — Refactoring: Improving the Design of Existing Code, 2nd ed.
- OpenAPI Specification 3.1.0 — <https://spec.openapis.org/oas/v3.1.0>
- Prisma ORM Documentation v5 — <https://www.prisma.io/docs>

2. Principios y Patrones de Diseño

2.1 Principios SOLID Aplicados

El diseño de todos los módulos del backend sigue los principios SOLID, que garantizan mantenibilidad, extensibilidad y testabilidad del código:

Principio	Aplicación en SmartCow	Ejemplo Concreto
S — Single Responsibility	Cada clase/módulo tiene una única razón para cambiar.	AlertEngine solo evalúa reglas. NotificationService solo envía notificaciones. Son clases separadas.
O — Open/Closed	Módulos abiertos para extensión, cerrados para modificación.	Agregar un nuevo tipo de alerta no modifica AlertEngine; se extiende implementando IAlertRule.
L — Liskov Substitution	Las implementaciones son intercambiables por sus interfaces.	EmailNotifier y SMSNotifier implementan INotifier; el motor de alertas no conoce la implementación concreta.
I — Interface Segregation	Interfaces pequeñas y específicas, no monolíticas.	IAnimalReader y IAnimalWriter separadas; el servicio de lectura no depende de métodos de escritura.
D — Dependency Inversion	Módulos de alto nivel no dependen de implementaciones concretas.	GPSService depende de IAnimalRepository (interfaz), no de PrismaAnimalRepository (concreción). Facilita mocking en tests.

2.2 Patrones de Diseño Aplicados (GoF)

Patrón	Categoría	Aplicación en SmartCow	Módulo
Repository	Estructural	Abstrae el acceso a PostgreSQL via Prisma. Los servicios no conocen SQL.	<code>src/*/repositories/</code>
Strategy	Comportamiento	Cada regla de alerta es una estrategia intercambiable (IAlertRule). El motor itera y evalúa.	<code>src/alerts/rules/</code>
Observer	Comportamiento	Los clientes WebSocket observan eventos de Redis	<code>src/realtime/</code>

		pub/sub. El GPS Service es el sujeto.	
Factory	Creacional	NotificationFactory crea el notificador adecuado (Email/SMS) según nivel de alerta y preferencias.	<code>src/notifications/</code>
Decorator	Estructural	Los middlewares de Express (auth, logger, rateLimit) decoran las rutas sin modificarlas.	<code>src/common/middlewares/</code>
Command	Comportamiento	Los trabajos de Bull MQ son comandos serializables (GenerateReportCommand, PersistGPSCommand).	<code>src/workers/commands/</code>
Singleton	Creacional	PrismaService y RedisService son singletons gestionados por el contenedor de dependencias.	<code>src/prisma/</code> , <code>src/redis/</code>
Circuit Breaker	Resiliencia	Las llamadas al servicio IA Python tienen un circuit breaker: si falla 3 veces, el sistema continúa sin predicciones.	<code>src/ai/ai.client.ts</code>

2.3 Arquitectura en Capas del Backend

El backend sigue una arquitectura en 4 capas con dependencias estrictamente unidireccionales (de afuera hacia adentro), inspirada en Clean Architecture:

Capa 1 — Presentación (Routes/Controllers): Recibe peticiones HTTP/WebSocket, valida DTOs de entrada con Zod, delega al servicio correspondiente, formatea y devuelve la respuesta. No contiene lógica de negocio.

Capa 2 — Aplicación (Services): Orquesta casos de uso: coordina repositorios, servicios externos y entidades. Contiene las reglas de negocio de alto nivel. Independiente de Express y Prisma.

Capa 3 — Dominio (Entities / Domain Services): Entidades puras del dominio ganadero (Animal, Geocerca, Alerta, SignoVital). Sin dependencias de frameworks. Contiene las reglas de negocio inmutables.

Capa 4 — Infraestructura (Repositories / Adapters): Implementaciones concretas: PrismaAnimalRepository, RedisGPSRepository, SendGridNotifier, TwilioNotifier. Dependen de librerías externas.

3. Diseño Detallado de Módulos

3.1 Módulo de Autenticación (auth)

3.1.1 Responsabilidades

- Gestionar el ciclo de vida de sesiones: login, logout, refresh token, 2FA.
- Generar y validar JSON Web Tokens (JWT) firmados con RS256 (par de claves RSA).
- Implementar bloqueo de cuentas tras intentos fallidos.
- Gestionar el segundo factor de autenticación TOTP (RFC 6238).

3.1.2 Interfaces Principales

```
// Contrato del servicio de autenticación
interface IAuthService {
    login(dto: LoginDto): Promise<AuthTokensDto>;
    logout(userId: string, refreshToken: string): Promise<void>;
    refreshTokens(refreshToken: string): Promise<AuthTokensDto>;
    verifyTwoFactor(userId: string, code: string): Promise<boolean>;
    generateTwoFactorSecret(userId: string): Promise<TwoFactorSetupDto>;
}

// DTOs de entrada/salida
interface LoginDto { email: string; password: string; }
interface AuthTokensDto { accessToken: string; refreshToken: string;
    expiresIn: number; }
interface TwoFactorSetupDto { secret: string; qrCodeUrl: string; }
```

3.1.3 Algoritmo de Autenticación

1. Recibir email + password via POST /api/v1/auth/login.
2. Buscar usuario en DB por email. Si no existe, retornar HTTP 401 (respuesta idéntica para no revelar existencia).
3. Verificar intentos fallidos en Redis: key auth:attempts:{userId}. Si ≥ 5 , retornar HTTP 429 con Retry-After header.
4. Comparar password con bcrypt.compare(plain, hash). Si falla, incrementar contador en Redis (TTL 15 min). Retornar HTTP 401.
5. Si 2FA activo: retornar HTTP 202 con challengeToken temporal (JWT de 5 min, scope: 2fa-pending). El cliente debe enviar el código TOTP.
6. Verificar código TOTP con speakeasy.totp.verify(). Ventana de ± 1 intervalo (30s) para tolerar desfase de reloj.
7. Generar accessToken JWT (RS256, 8h) con payload: { sub: userId, farmId, role, permissions }. Generar refreshToken (UUID v4, SHA-256 hash almacenado en DB, TTL 30d).
8. Retornar accessToken en body JSON. Establecer refreshToken en cookie HttpOnly, Secure, SameSite=Strict.

3.1.4 Middleware de Autorización RBAC

```
// Tabla de permisos por rol
const PERMISSIONS = {
  SUPER_ADMIN: ['*'], // Todos los permisos
  ADMIN:
  ['animals:*','geofences:*','alerts:*','users:read','users:write','reports:*'],
  VET:
  ['animals:read','animals:health:write','alerts:read','alerts:write'],
  OPERATOR: ['animals:read','alerts:read','alerts:acknowledge'],
};

// Uso en rutas
router.delete('/animals/:id',
  authenticate, // Valida JWT
  authorize('animals:*'), // Verifica permiso
  AnimalController.delete
);
```

3.2 Módulo GPS y Telemetría IoT (gps)

3.2.1 Contrato del Payload IoT

```
// Payload JSON que envía cada collar IoT via MQTT
interface IoTTelemetryPayload {
  deviceId: string; // UUID del collar (configurado en flash del ESP32)
  timestamp: number; // Unix timestamp en ms (UTC)
  gps: {
    lat: number; // Latitud decimal (WGS84)
    lng: number; // Longitud decimal (WGS84)
    alt: number; // Altitud en metros
    accuracy: number; // Precisión en metros (HDOP * 5)
    speed: number; // Velocidad en km/h
  };
  vitals: {
    tempC: number; // Temperatura corporal en °C (DS18B20)
    heartBpm: number; // Ritmo cardíaco en bpm
    activity: number; // Índice 0-100 (magnitud acelerómetro normalizada)
  };
  battery: number; // Nivel batería 0-100%
  firmwareVer: string; // Versión firmware del collar
}
```

3.2.2 Pipeline de Procesamiento GPS

Paso	Operación	Implementación	Salida
1	Validación de esquema	<code>zod.parse(payload)</code>	Payload validado o error 400
2	Resolución animal-dispositivo	Redis HGET <code>device:{deviceId}</code>	animalId + farmId
3	Actualizar posición actual en cache	Redis HSET <code>animal:{id}</code> <code>lat lng ts</code>	Cache actualizado (< 2ms)
4	Publicar en canal tiempo real	Redis PUBLISH <code>farm:{farmId}</code>	Evento emitido a clientes WS

5	Encolar persistencia asíncrona	Bull MQ: persist-gps queue	Job en cola (no bloqueante)
6	Encolar evaluación de alertas	Bull MQ: evaluate-alert queue	Job de evaluación en cola
7	Worker: persistir en PostgreSQL	INSERT INTO gps_history	Registro histórico persistido
8	Worker: evaluar reglas de alerta	AlertEngine.evaluate(data)	Alertas generadas si aplica

3.3 Módulo de Geocercas (geofence)

3.3.1 Algoritmo Point-in-Polygon

Para determinar si un animal está dentro o fuera de una geocerca poligonal, se implementa el algoritmo Ray Casting (Jordan curve theorem). Este algoritmo lanza un rayo horizontal desde el punto a evaluar y cuenta cuántas veces cruza los bordes del polígono.

```
// Algoritmo Ray Casting - O(n) donde n = número de vértices
function isPointInPolygon(point: LatLng, polygon: LatLng[]): boolean {
  const { lat: px, lng: py } = point;
  let inside = false;
  const n = polygon.length;
  for (let i = 0, j = n - 1; i < n; j = i++) {
    const xi = polygon[i].lat, yi = polygon[i].lng;
    const xj = polygon[j].lat, yj = polygon[j].lng;
    const intersect =
      yi > py !== yj > py &&
      px < ((xj - xi) * (py - yi)) / (yj - yi) + xi;
    if (intersect) inside = !inside;
  }
  return inside;
}
```

Optimización: Las geocercas activas se cachean en Redis al inicio y se invalidan al modificarse. El lookup por farmId en Redis es $O(1)$ antes de ejecutar el algoritmo $O(n)$ de polígono.

3.3.2 Diseño del GeofenceService

```
interface IGeofenceService {
  // Evalúa si un animal cruzó alguna geocerca activa
  evaluate(animalId: string, point: LatLng, farmId: string):
  Promise<GeofenceEvent[]>;

  // CRUD de geocercas
  create(dto: CreateGeofenceDto, userId: string): Promise<Geofence>;
  update(id: string, dto: UpdateGeofenceDto): Promise<Geofence>;
  delete(id: string): Promise<void>;
  findByFarm(farmId: string): Promise<Geofence[]>;

  // Invalida cache de Redis para una finca
  invalidateCache(farmId: string): Promise<void>;
}
```



```
interface GeofenceEvent {
  geofenceId: string;
  animalId: string;
  type: 'ENTER' | 'EXIT';
  timestamp: Date;
  coordinates: LatLng;
}
```

3.4 Módulo Motor de Alertas (alerts)

3.4.1 Arquitectura de Reglas — Patrón Strategy

El motor de alertas implementa el patrón Strategy para que cada tipo de alerta sea una regla independiente, evaluable y testeable de forma aislada. Agregar una nueva regla solo requiere implementar la interfaz IAlertRule.

```
// Interfaz base para todas las reglas de alerta
interface IAlertRule {
  readonly ruleId: string;
  readonly priority: AlertPriority; // CRITICAL | WARNING | INFO
  evaluate(context: AlertContext): AlertResult | null;
}

interface AlertContext {
  animal: Animal;
  telemetry: IoTTelemetryPayload;
  geofences: Geofence[];
  thresholds: FarmThresholds;
  lastPosition?: LatLng;
}

// Implementación de regla: temperatura alta
class HighTemperatureRule implements IAlertRule {
  readonly ruleId = 'HIGH_TEMP';
  readonly priority = AlertPriority.WARNING;

  evaluate(ctx: AlertContext): AlertResult | null {
    const { tempC } = ctx.telemetry.vitals;
    const { tempWarn, tempCritical } = ctx.thresholds;
    if (tempC >= tempCritical)
      return { priority: AlertPriority.CRITICAL, message: `Temperatura crítica: ${tempC}°C` };
    if (tempC >= tempWarn)
      return { priority: AlertPriority.WARNING, message: `Temperatura elevada: ${tempC}°C` };
    return null;
  }
}
```

3.4.2 Reglas de Alerta Implementadas

Clase de Regla	Prioridad	Condición de Disparo	Notificación
GeofenceExitRule	CRITICAL	Coordenadas GPS fuera del polígono de geocerca activa.	Web + Email + SMS

HighTemperatureRule	WARNING/CRITICAL	$\geq 39.5^{\circ}\text{C} \rightarrow \text{WARNING.}$ $\geq 40.5^{\circ}\text{C} \rightarrow \text{CRITICAL.}$	Web + Email (WARN). + SMS (CRIT)
AbnormalHeartRateRule	WARNING/CRITICAL	$< 40 \text{ bpm o } > 100 \text{ bpm} \rightarrow \text{WARNING.}$ $< 30 \text{ o } > 120 \rightarrow \text{CRITICAL.}$	Web + Email (WARN). + SMS (CRIT)
SensorOfflineRule	WARNING	Sin telemetría > 5 minutos. Evaluado por job periódico.	Web + Email
ProlongedImmobilityRule	WARNING	Actividad < 5 (índice) durante > 4 horas continuas.	Web + Email
NightMovementRule	CRITICAL	Velocidad > 15 km/h entre 22:00-05:00 fuera de geocerca.	Web + Email + SMS
LowBatteryRule	INFO	Nivel de batería del collar < 15%.	Web únicamente

3.5 Módulo de Inventario de Ganado (animals)

3.5.1 Diseño del AnimalService

```
interface IAnimalService {
  create(dto: CreateAnimalDto, farmId: string): Promise<Animal>;
  findById(id: string, farmId: string): Promise<AnimalDetailDto>;
  findAll(query: AnimalQueryDto, farmId: string):
  Promise<PaginatedResult<AnimalSummaryDto>>;
  update(id: string, dto: UpdateAnimalDto, farmId: string): Promise<Animal>;
  delete(id: string, farmId: string): Promise<void>;
  importFromCsv(file: Buffer, farmId: string): Promise<ImportResultDto>;
  exportToCsv(farmId: string): Promise<Buffer>;
  getLocationHistory(id: string, from: Date, to: Date): Promise<GPSPoint[]>;
  getVitalHistory(id: string, from: Date, to: Date): Promise<VitalRecord[]>;
}

// Respuesta paginada estándar para todos los listados
interface PaginatedResult<T> {
  data: T[];
  total: number;
  page: number;
  pageSize: number;
  totalPages: number;
}
```

3.5.2 Algoritmo de Importación CSV

9. Recibir archivo CSV (multipart/form-data, máx. 5MB).
10. Parsear con papaparse en modo streaming para soportar archivos grandes.
11. Validar cada fila con Zod schema. Acumular errores por número de fila.
12. Si hay errores de validación, rechazar toda la importación y retornar reporte de errores.
13. Si no hay errores, ejecutar INSERT masivo via prisma.createMany() en una transacción.
14. Generar y retornar reporte: total procesados, insertados, omitidos (duplicados por código), errores.

3.6 Módulo de Analítica e IA (analytics / ai)

3.6.1 Diseño del Servicio de Predicción

```
// Contrato con el servicio Python de ML
interface IAIClient {
    predictHealth(input: HealthPredictionInput): Promise<HealthPredictionOutput>;
}

interface HealthPredictionInput {
    animalId: string;
    // Series de tiempo de las últimas 72 horas (array de registros cada 10 min)
    vitals: Array<{
        timestamp: number;
        tempC: number;
        heartBpm: number;
        activity: number;
    }>;
}

interface HealthPredictionOutput {
    riskScore: number; // 0.0 - 1.0 (probabilidad de evento de salud en 48h)
    riskLevel: 'LOW' | 'MEDIUM' | 'HIGH';
    confidence: number; // 0.0 - 1.0
    topFeatures: Array<{ feature: string; importance: number }>;
    recommendation: string; // Acción sugerida en texto
}
```

3.6.2 Modelo de Machine Learning

El modelo de predicción de salud es un clasificador de series de tiempo implementado en Python con scikit-learn. El pipeline de ML consiste en:

15. Preprocesamiento: normalización Z-score de señales vitales, interpolación de valores faltantes, extracción de features estadísticas (media, desviación, tendencia, cruces por cero en ventanas de 1h, 6h, 24h).
16. Modelo base: Random Forest Classifier con 200 árboles. Entrenado con datos históricos etiquetados de eventos de salud. Umbral de clasificación ajustable por finca (calibración por raza).
17. Salida: probabilidad de evento de salud en las próximas 24-48 horas con importancia de features para explicabilidad (SHAP values).
18. Reentrenamiento: job semanal automatizado con nuevos datos etiquetados. Modelos versionados con MLflow (futuro).

4. Diseño de la API REST

4.1 Convenciones Generales

- Base URL: `https://api.smartcow.app/api/v1`
- Autenticación: Bearer token JWT en header `Authorization` para todos los endpoints excepto `/auth/login`.
- Content-Type: `application/json` en todas las solicitudes y respuestas.
- Paginación estándar: parámetros `?page=1&pageSize=20` en todos los listados.
- Formato de fechas: ISO 8601 (`2026-05-28T14:30:00Z`) en todos los campos de tipo fecha.
- Errores: estructura estándar `{ statusCode, error, message, details? }` en todos los errores.
- Versionado: la versión de la API se incluye en la URL (`/api/v1/`). Cambios breaking generan nueva versión.

4.2 Endpoints por Módulo

4.2.1 Autenticación (/auth)

Método	Endpoint	Autenticación	Descripción
POST	<code>/auth/login</code>	Ninguna	Inicio de sesión. Devuelve JWT + establece cookie refresh.
POST	<code>/auth/logout</code>	JWT requerido	Cierre de sesión. Invalida refresh token.
POST	<code>/auth/refresh</code>	Cookie refresh	Renueva el access token usando el refresh token.
POST	<code>/auth/2fa/verify</code>	Challenge token	Verifica código TOTP del segundo factor.
GET	<code>/auth/2fa/setup</code>	JWT requerido	Genera secreto TOTP y QR code para configuración.

4.2.2 Ganado (/animals)

Método	Endpoint	Rol mínimo	Descripción
GET	<code>/animals</code>	OPERATOR	Lista paginada con filtros: <code>?status=&breed=&sector=&search=</code>
POST	<code>/animals</code>	ADMIN	Registrar nuevo animal. Body: <code>CreateAnimalDto</code> .

GET	<code>/animals/:id</code>	OPERATOR	Ficha completa: datos + vitales actuales + historial médico.
PATCH	<code>/animals/:id</code>	ADMIN/VET	Actualización parcial de datos del animal.
DELETE	<code>/animals/:id</code>	ADMIN	Baja lógica del animal (soft delete, conserva historial).
GET	<code>/animals/:id/location</code>	OPERATOR	Posición GPS actual desde Redis cache.
GET	<code>/animals/:id/history</code>	OPERATOR	Historial GPS: ?from=&to= (máx 30 días).
GET	<code>/animals/:id/vitals</code>	OPERATOR	Historial de signos vitales: ?from=&to=&resolution=
GET	<code>/animals/:id/prediction</code>	VET	Predicción de salud del modelo IA para este animal.
POST	<code>/animals/import</code>	ADMIN	Importación masiva CSV. Content-Type: multipart/form-data.
GET	<code>/animals/export</code>	ADMIN	Exportar inventario completo en CSV o XLSX. ?format=csv xlsx

4.2.3 Geocercas (/geofences)

Método	Endpoint	Rol mínimo	Descripción
GET	<code>/geofences</code>	OPERATOR	Lista todas las geocercas de la finca con sus polígonos.
POST	<code>/geofences</code>	ADMIN	Crear geocerca. Body: { name, polygon: LatLng[], color, active }.
PATCH	<code>/geofences/:id</code>	ADMIN	Actualizar nombre, polígono, color o estado activo/inactivo.
DELETE	<code>/geofences/:id</code>	ADMIN	Eliminar geocerca. Requiere confirmación si tiene alertas activas.

4.2.4 Alertas (/alerts)

Método	Endpoint	Rol mínimo	Descripción
GET	<code>/alerts</code>	OPERATOR	Lista alertas con filtros: ?status=OPEN&priority=CRITICAL&animalId=

GET	/alerts/:id	OPERATOR	Detalle completo de una alerta con historial de acciones.
POST	/alerts/:id/acknowledge	OPERATOR	Marcar alerta como reconocida (ACKNOWLEDGED). Registra usuario y timestamp.
POST	/alerts/:id/assign	ADMIN	Asignar alerta a un operador. Body: { assigneeId: string }.
POST	/alerts/:id/close	OPERATOR	Cerrar alerta. Body: { resolution: string }. Estado → CLOSED.

4.3 Eventos WebSocket

El servidor emite eventos WebSocket a los clientes conectados. El cliente se suscribe al namespace de su finca: /farms/{farmId}.

Evento	Payload	Descripción
animal:position	{ animalId, lat, lng, ts }	Nueva posición GPS de un animal. Emitido cada 10 segundos por animal activo.
animal:vitals	{ animalId, tempC, heartBpm, activity }	Actualización de signos vitales. Acompaña a cada actualización de posición.
alert:new	{ alertId, priority, type, animalId }	Nueva alerta generada. El cliente muestra notificación inmediata.
alert:updated	{ alertId, status, updatedBy }	Cambio de estado de alerta (reconocida, asignada, cerrada).
sensor:offline	{ animalId, deviceId, lastSeen }	Sensor dejó de transmitir por más de 5 minutos.
sensor:online	{ animalId, deviceId }	Sensor volvió a estar en línea después de un periodo offline.

5. Diseño de Base de Datos

5.1 Esquema Prisma — Entidades Principales

El esquema de base de datos está definido en Prisma Schema Language (PSL). A continuación se describen las entidades principales y sus relaciones.

```
// Entidad: Finca (tenant principal del sistema)
model Farm {
  id          String      @id @default(cuid())
  name        String
  slug        String      @unique
  country     String      @default('CO')
  timezone    String      @default('America/Bogota')
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  animals     Animal[]
  users       UserFarm[]
  geofences   Geofence[]
  thresholds  FarmThresholds?
  alerts      Alert[]
}

// Entidad: Animal (cabeza de ganado)
model Animal {
  id          String      @id @default(cuid())
  farmId      String
  code        String      // Código único en la finca
  name        String?
  breed       String
  sex         Sex          // MALE | FEMALE
  birthDate   DateTime?
  weightKg    Float?
  photoUrl    String?
  status      AnimalStatus @default(ACTIVE) // ACTIVE | INACTIVE | SOLD | DECEASED
  healthStatus HealthStatus @default(HEALTHY) // HEALTHY | ALERT | CRITICAL
  deviceId    String?      @unique           // ID del collar IoT asignado
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  farm        Farm         @relation(fields:[farmId], references:[id])
  gpsHistory  GpsHistory[]
  vitalHistory VitalHistory[]
  alerts      Alert[]
  healthEvents HealthEvent[]
  @@unique([farmId, code])
  @@index([farmId, status])
}

// Entidad: Historial GPS (serie de tiempo de posiciones)
model GpsHistory {
  id          BigInt      @id @default(autoincrement())
  animalId    String
  farmId      String
  lat         Float
  lng         Float
  alt         Float?
```

```

    accuracy    Float?
    speedKmh    Float?
    recordedAt  DateTime
    animal      Animal    @relation(fields:[animalId], references:[id])
    @@index([animalId, recordedAt(sort: Desc)])
    @@index([farmId, recordedAt(sort: Desc)])
  }

// Entidad: Signos Vitales históricos
model VitalHistory {
  id          BigInt      @id @default(autoincrement())
  animalId    String
  farmId      String
  tempC       Float
  heartBpm    Int
  activity     Int
  batteryPct  Int
  recordedAt  DateTime
  animal      Animal    @relation(fields:[animalId], references:[id])
  @@index([animalId, recordedAt(sort: Desc)])
}

// Entidad: Geocerca
model Geofence {
  id          String      @id @default(cuid())
  farmId      String
  name        String
  description  String?
  // Polígono almacenado como JSON array de {lat, lng}
  polygon     Json
  color       String      @default('#1a7a4a')
  active      Boolean      @default(true)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  farm        Farm        @relation(fields:[farmId], references:[id])
  alerts      Alert[]
  @@index([farmId, active])
}

```

```

// Entidad: Alerta
model Alert {
  id          String      @id @default(cuid())
  farmId      String
  animalId    String
  geofenceId  String?
  ruleId      String      // Identificador de la regla
  // que disparó
  priority    AlertPriority // CRITICAL | WARNING | INFO
  status      AlertStatus  @default(OPEN) // OPEN | ACKNOWLEDGED |
  // ASSIGNED | CLOSED
  title       String
  description  String
  metadata     Json?      // Datos adicionales del
  // evento
  assigneeId  String?
  resolution  String?
  openedAt    DateTime    @default(now())
  closedAt    DateTime?
  responseMs  Int?        // Tiempo de respuesta en
  // milisegundos
  farm        Farm        @relation(fields:[farmId], references:[id])
  animal      Animal      @relation(fields:[animalId], references:[id])
}

```



```

geofence    Geofence?    @relation(fields:[geofenceId], references:[id])
@@index([farmId, status, priority])
@@index([animalId, openedAt(sort: Desc)])
}

// Entidad: Usuario
model User {
  id          String      @id @default(cuid())
  email       String      @unique
  passwordHash String
  name        String
  phone       String?
  twoFactorSecret String?
  twoFactorEnabled Boolean @default(false)
  loginAttempts Int        @default(0)
  lockedUntil DateTime?
  createdAt   DateTime     @default(now())
  farms       UserFarm[]
  auditLogs   AuditLog[]
}

// Relación Usuario-Finca con rol
model UserFarm {
  userId  String
  farmId  String
  role    UserRole // SUPER_ADMIN | ADMIN | VET | OPERATOR
  user    User      @relation(fields:[userId], references:[id])
  farm    Farm      @relation(fields:[farmId], references:[id])
  @@id([userId, farmId])
}

```

5.2 Estrategia de Índices y Rendimiento

Las tablas GpsHistory y VitalHistory son las de mayor volumen (247 animales × 6 registros/min = ~1,500 registros/min). La estrategia de gestión es:

- Índices compuestos en (animalId, recordedAt DESC) para consultas de historial individual.
- Índices compuestos en (farmId, recordedAt DESC) para consultas de toda la finca.
- Particionamiento por rango de fecha en GpsHistory y VitalHistory (particiones mensuales) para mantener el rendimiento de consultas a medida que los datos crecen.
- Retención de datos: registros con antigüedad > 2 años se archivan a cold storage (S3). Job mensual automatizado con pg_partman.
- Posición actual (dato más consultado): NO se lee de GpsHistory. Se lee directamente de Redis HSET para latencia < 2ms.

5.3 Estrategia de Migración

- Todas las migraciones de esquema se gestionan con prisma migrate. Cada migración genera un archivo SQL versionado en prisma/migrations/.
- Migraciones destructivas (DROP COLUMN, DROP TABLE) requieren aprobación explícita del Tech Lead y se ejecutan en ventana de mantenimiento.
- En CI/CD, la pipeline ejecuta prisma migrate deploy antes de iniciar el servidor en cada despliegue a staging y production.

6. Diseño del Frontend (React SPA)

6.1 Gestión de Estado

El frontend usa dos librerías complementarias para gestión de estado, cada una en su dominio óptimo:

Librería	Caso de Uso	Datos Gestionados
Redux Toolkit	Estado global de tiempo real. Se actualiza via WebSocket.	Posiciones GPS actuales de todos los animales. Alertas activas en tiempo real. Estado de conexión WebSocket.
TanStack Query (React Query)	Cache de datos del servidor. Datos obtenidos via API REST.	Ficha de animal, historial de rutas, listados, datos de analytics, configuración de geocercas.

Regla de diseño: Si el dato cambia en tiempo real via WebSocket → Redux. Si el dato se obtiene con HTTP GET y puede cachearse → React Query. Nunca duplicar el mismo dato en ambas librerías.

6.2 Componentes del Mapa (MapView)

El componente MapView es el de mayor complejidad del frontend. Su diseño contempla:

- Uso de React-Leaflet como wrapper declarativo de Leaflet.js para integración con el árbol de React.
- AnimalMarkerLayer: componente memoizado que renderiza los marcadores de todos los animales. Usa React.memo y compara solo lat/lng para evitar re-renders innecesarios.
- GeofenceLayer: renderiza polígonos de geocercas usando L.polygon. Los polígonos se cargan una vez al montar y se actualizan solo cuando el usuario modifica una geocerca.
- WebSocket subscription: useEffect en MapView se suscribe al evento animal:position de Redux. Solo actualiza marcadores con posición cambiada (diff por animalId).
- Rendimiento con 247 marcadores: se usa VirtualizedMarkerLayer para renderizar solo los marcadores dentro del viewport visible + un buffer de 20% alrededor.

6.3 Componente VitalCards (Tiempo Real)

```
// Hook personalizado para suscripción en tiempo real a vitales
function useAnimalVitals(animalId: string) {
  const dispatch = useDispatch();
  const vitals = useSelector(selectAnimalVitals(animalId));

  useEffect(() => {
    // Suscribirse al evento de vitales via WebSocket
    socket.on(`animal:vitals:${animalId}`, (data: VitalUpdate) => {
      dispatch(updateAnimalVitals({ animalId, ...data }));
    });
  });
}
```

```
    return () => socket.off(`animal:vitals:${animalId}`);
  }, [animalId, dispatch]);

  return vitals;
}

// Mini-gráfico de tendencia con últimas 5 lecturas
function VitalCard({ animalId, metric }: VitalCardProps) {
  const vitals = useAnimalVitals(animalId);
  const history = useVitalHistory(animalId, metric, 5); // React Query
  return (
    <div className='vital-card'>
      <span className='value'>{vitals[metric]}</span>
      <MiniSparkline data={history} />
    </div>
  );
}
```

6.4 Modo Offline

El modo offline del frontend garantiza que los datos más recientes estén disponibles sin conexión:

19. Service Worker (Workbox) cachea el shell de la aplicación (index.html, chunks JS/CSS) con estrategia Cache First.
20. Las últimas respuestas de API REST se cachean en IndexedDB usando la estrategia Network First con fallback a cache.
21. Los datos ingresados sin conexión (actualizaciones de campos del animal, notas) se almacenan en una cola offline en IndexedDB.
22. Al restaurar la conexión, el Service Worker detecta la reconexión (online event), procesa la cola offline y sincroniza con el servidor.
23. Conflictos de sincronización: último timestamp gana (last-write-wins). El servidor registra un log de sincronización offline.

7. Diseño de Seguridad

7.1 Modelo de Amenazas (STRIDE)

Amenaza STRIDE	Superficie de Ataque	Control de Mitigación	Referencia
Spoofing	Login de usuarios, tokens IoT.	JWT RS256 + 2FA obligatorio para admins. Certificados de cliente para collares IoT.	RF-AUTH-001, RF-AUTH-002
Tampering	Payload IoT, requests API.	TLS 1.3 extremo a extremo. Validación de esquema Zod en cada endpoint. Firma JWT.	RNF-SE-001
Repudiation	Acciones críticas del sistema.	Log de auditoría inmutable con usuario, IP, timestamp, payload diferencial en PostgreSQL.	RF-AUTH-004
Info Disclosure	Respuestas de API, logs.	RBAC estricto. Filtro por farmId en todas las queries. Sin stack traces en producción. Secrets en variables de entorno.	RNF-SE-006
Denial of Service	Endpoints públicos (/auth/login).	Rate limiting (100 req/min por IP) en Nginx + Express. Cloudflare WAF como primera capa.	RNF-SE-005
Elevation of Privilege	Endpoints administrativos.	Middleware authorize() verifica permisos en cada ruta. Tokens JWT no contienen permisos modificables por el cliente.	RF-AUTH-003

7.2 Encriptación de Datos en Reposo

- Contraseñas: bcrypt con salt único por contraseña y factor de costo 12. Nunca almacenadas en texto plano.
- Secretos 2FA (TOTP): encriptados con AES-256-GCM usando una clave maestra almacenada en variables de entorno (nunca en base de datos).
- Datos de telemetría IoT: almacenados sin encriptación adicional a nivel de fila (el transporte usa TLS). Encriptación a nivel de disco (database encryption at rest) gestionada por el proveedor cloud.
- Backups de base de datos: encriptados con AES-256 antes de subir a S3.
- Variables de entorno: gestionadas con el secret manager del proveedor cloud (Railway Secrets / AWS Secrets Manager). Nunca en repositorio de código.

8. Estrategia de Pruebas

8.1 Pirámide de Pruebas

La estrategia de pruebas sigue la pirámide clásica de Mike Cohn, con mayor énfasis en pruebas unitarias y de integración por su velocidad de ejecución y valor de detección de regresiones:

Nivel	Herramienta	Cobertura objetivo	Módulos prioritarios
Unitarias	Vitest / Jest	$\geq 80\%$	AlertEngine (todas las reglas), AuthService, GeofenceService (algoritmo P-i-P), GPSService pipeline.
Integración API	Supertest + Docker	$\geq 60\%$	Todos los endpoints REST. BD real en contenedor Docker para pruebas.
Componentes UI	Vitest + Testing Library	$\geq 50\%$	MapView, VitalCards, AlertItem, AnimalCard, LoginForm.
E2E (flujos críticos)	Playwright	5 flujos clave	Login → Dashboard, Ver mapa → Click animal, Crear geocerca, Reconocer alerta, Importar CSV.

8.2 Casos de Prueba Críticos

- TEST-AUTH-001: Login con credenciales válidas → JWT retornado, cookie establecida.
- TEST-AUTH-002: 5 intentos fallidos → cuenta bloqueada, HTTP 429 en 6to intento.
- TEST-GEO-001: Punto dentro del polígono → isPointInPolygon() retorna true (100 casos generados aleatoriamente).
- TEST-GEO-002: Punto fuera del polígono → isPointInPolygon() retorna false.
- TEST-ALERT-001: Temperatura 40.6°C → HighTemperatureRule genera alerta CRITICAL.
- TEST-ALERT-002: Temperatura 38.2°C → HighTemperatureRule retorna null (sin alerta).
- TEST-GPS-001: Payload MQTT válido procesado en $< 100\text{ms}$ en el GPS Service.
- TEST-RBAC-001: Usuario OPERATOR intenta DELETE /animals/:id → HTTP 403.
- TEST-TENANT-001: JWT de farmId-A no puede acceder a recursos de farmId-B → HTTP 403.

Apéndice A: Códigos de Error Estándar de la API

HTTP	Código interno	Descripción	Cuándo ocurre
400	VALIDATION_ERROR	Error de validación del body o query params.	DTO inválido, campo faltante o formato incorrecto.
401	UNAUTHORIZED	No autenticado o token inválido/expirado.	JWT ausente, expirado o firma inválida.
403	FORBIDDEN	Sin permisos para la acción solicitada.	Rol insuficiente o intento de acceso cross-tenant.
404	NOT_FOUND	Recurso no encontrado.	animal, geofence, o alert con ID inexistente.
409	CONFLICT	Conflicto con el estado actual del recurso.	Código de animal duplicado en la finca.
429	RATE_LIMITED	Demasiadas solicitudes.	Supera 100 req/min en /auth o cuenta bloqueada.
500	INTERNAL_ERROR	Error interno del servidor.	Excepción no manejada. Se registra en logs con stacktrace.
503	SERVICE_UNAVAILABLE	Servicio externo no disponible.	Servicio IA Python caído (circuit breaker abierto).

Apéndice B: Variables de Entorno Requeridas

Variable	Ejemplo	Descripción
DATABASE_URL	postgresql://...	URL de conexión a PostgreSQL con credenciales.
REDIS_URL	redis://...	URL de conexión a Redis.
JWT_PRIVATE_KEY	-----BEGIN RSA...	Clave privada RSA para firma de JWT (RS256).
JWT_PUBLIC_KEY	-----BEGIN PUBLIC...	Clave pública RSA para verificación de JWT.
TWO_FACTOR_ENCRYPTION_KEY	32-char hex string	Clave AES-256 para encriptar secretos TOTP.
MQTT_BROKER_URL	mqtt://broker:8883	URL del broker MQTT con protocolo MQTTS.

SENDGRID_API_KEY	SG.xxxxxx	Clave API de SendGrid para envío de emails.
TWILIO_ACCOUNT_SID	ACxxxxxx	SID de cuenta Twilio para envío de SMS.
TWILIO_AUTH_TOKEN	xxxxxxxxxx	Token de autenticación Twilio.
AI_SERVICE_URL	http://ai:8000	URL interna del servicio Python de ML.
STORAGE_BUCKET	smartcow-media	Nombre del bucket S3 para fotos y reportes.
NODE_ENV	production	Entorno de ejecución: development staging production.

— Fin del Documento SDD v1.0.0 —

SmartCow Tracker | Documento confidencial | Mayo 2026